PROJECT REPORT

Implement Client Script & UI Policy (Incident)

Team ID: SWTID-2026-6870

Team Size: 4

Team Leader: HARISH V

Team Member: Jairaj R

Team Member: KARTHIKEYAN S

Team Member: KARUPPUSAMY PRAVEEN N

1. INTRODUCTION

1.1 Project Overview

This project implements Client Scripts and UI Policies on the ServiceNow Incident table to improve data integrity and control user interaction with Incident records. The main idea is to make the Incident form respond automatically when users select important values such as Impact.

In a normal Incident-management process, users enter information manually. If every field is left completely open, users may accidentally leave important fields empty, enter values in an unsuitable combination, or change a field from a list view without following the intended process. The project addresses these situations at the point of user interaction.

The solution is intentionally simple. UI Policies are used for field behavior such as mandatory and read-only conditions, while Client Scripts are used when a specific action or validation must occur. The rules work on the Incident form and, where required, on Incident list editing.

The project focuses on High Impact Incidents because these records normally require greater attention and more reliable information. When Impact is set to High, the system increases control over Assignment group and Urgency. Additional validation checks Assigned To before the record is saved.

The project also demonstrates that the Incident form and Incident list can be controlled differently. A State change is prevented when it is attempted through list editing, while a State change made through the normal Incident form remains allowed. This distinction provides a controlled user experience without unnecessarily restricting the form.

1.2 Purpose

The purpose is to reduce incomplete or inconsistent Incident records by dynamically controlling fields, automatically setting values, validating submissions, and preventing invalid list edits.

The project provides a practical example of client-side customization in ServiceNow. Instead of asking users to remember every rule, the application applies the rules automatically. This makes the process easier for users and gives administrators a more predictable data-entry environment.

Another purpose is to demonstrate how different ServiceNow client-side features can work together. A UI Policy handles visible field behavior, an onChange Client Script reacts to a field change, an

onSubmit Client Script checks information before saving, and an onCellEdit Client Script controls list editing.

2. IDEATION PHASE

2.1 Problem Statement

Incident records can become incomplete, inconsistent, or incorrect when important fields are not properly validated during manual entry. This can affect reporting, SLA compliance, and service quality.

The problem begins when the system depends only on users to remember which fields are important in each situation. For example, a High Impact Incident may require an Assignment group, but a user may not immediately notice that the field is empty. Similarly, Urgency may be expected to follow a defined business rule, but manual editing can produce a value that does not match the intended process.

A second problem is that form editing and list editing provide different ways of changing a record. If a State value can be changed freely from a list, a user may bypass the controlled form experience. The project therefore considers not only what happens on the form, but also what happens when a user edits the record from the list.

The project solution was selected because the required behavior is mainly related to user interaction. It does not require a separate application, external database, or custom REST service. The ServiceNow platform already provides the required table, form, list, client API, UI Policy mechanism, and Client Script mechanism.

2.2 Empathy Map Canvas

User needs identified from the project context:

Users need clear guidance when entering Incident information.

Users should be prevented from submitting incomplete High Impact Incidents.

Users need important values such as Urgency to be controlled consistently.

Administrators need reliable Incident data for reporting and service management.

Users need the form to provide immediate feedback instead of discovering an error later.

Administrators need rules that are easy to understand, test, and maintain.

Users should be able to continue normal Incident updates when the special High Impact condition is not active.

The empathy perspective shows that the project is not only about writing scripts. It is also about reducing confusion during Incident entry. A user should not have to remember every condition manually. The interface should communicate the condition through mandatory or read-only behavior, while validation should clearly explain why a submission is not accepted.

2.3 Brainstorming

Use UI Policies for dynamic field behavior.

Use Client Scripts for automatic field updates.

Validate required information before form submission.

Prevent inappropriate State changes through list editing.

Keep the solution inside the ServiceNow platform so that no separate application is required.

Test both positive and negative scenarios. A good implementation should not only block invalid actions; it should also allow valid actions and reverse temporary restrictions when the condition changes.

The brainstorming process therefore resulted in a combination of configuration and scripting rather than using one feature for every requirement. This keeps each rule understandable and makes the final demonstration easier.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

Stage	User Action	System Response
Create/Update Incident	Enter Incident information	Form accepts input and displays the available Incident fields.
Set High Impact	Change Impact to 1 – High	Assignment group becomes mandatory and Urgency becomes read-only.
High Impact automation	Impact changes to High	Urgency is automatically set to High.
Submit Incident	Attempt save without Assigned To	Submission is blocked with an error message.
Submit Incident	Assigned To is provided	Validation passes and the record can be saved.
List editing	Try to edit State from list	Alert appears and the edit is rejected.
Form update	Change State from Incident form	State update is allowed.
Change condition	High Impact is changed to another value	Temporary High Impact restrictions are reversed and Urgency becomes editable.

The customer journey shows the project from the user's point of view. The user starts with a normal Incident, changes the Impact, receives immediate field-level guidance, and then receives validation at save time if required information is still missing.

The journey also shows why multiple rules are required. A UI Policy alone can control the appearance and behavior of fields, but it does not replace the specific validation needed before

submission. Similarly, a Client Script that sets Urgency does not by itself make Assignment group mandatory. Each rule addresses a separate point in the journey.

3.2 Solution Requirement

UI Policy on Incident table with condition Impact = 1 – High.

UI Policy Action to make Assignment group mandatory.

UI Policy Action to make Urgency read-only.

onChange Client Script to set Urgency to High when Impact becomes High.

onSubmit Client Script to prevent saving a High Impact Incident when Assigned To is empty.

onCellEdit Client Script to prevent State changes through list editing.

The requirements are functional and user-facing. The expected result of each requirement can be observed directly in the ServiceNow interface.

The requirements also include an important reverse condition. When Impact is no longer High, the special restrictions should no longer remain active. This ensures that the UI behavior is conditional rather than permanently restrictive.

3.3 Data Flow Diagram

Conceptual flow:

User → Incident Form/List → UI Policy & Client Script Rules → Validation/Field Control → Incident Record

The data flow is represented as a conceptual text flow for the project.

The flow begins with the user. The user performs an action on the Incident form or list. ServiceNow evaluates the configured UI Policy or Client Script. If the condition is satisfied, the corresponding field behavior or validation is applied. If the action is valid, the Incident record continues through the normal ServiceNow process.

For example, when the user changes Impact to High, the onChange Client Script sets Urgency to High and the UI Policy makes Assignment group mandatory and Urgency read-only. When the user attempts to save without Assigned To, the onSubmit validation checks the record and stops the submission.

3.4 Technology Stack

Technology/Component	Purpose
ServiceNow	Development and execution platform.
Incident Table	Target table for the project rules.
UI Policy	Dynamic field behavior and mandatory/read-only control.
UI Policy Action	Applies field-level behavior.
Client Script	Client-side automation and validation.
JavaScript	Language used for Client Script logic.
g_form	ServiceNow client-side API used to read, set, and control form fields.

The technology stack is intentionally limited to platform features required by the project. No separate frontend framework, backend server, or external database is required.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution directly addresses incomplete Incident data by applying conditional field controls and validation at the point where users create or update Incident records.

The fit between problem and solution is clear. The problem is caused by incorrect or incomplete user input, so the solution operates close to the input itself. A UI Policy changes field behavior when Impact becomes High. An onChange Client Script provides automatic value setting. An onSubmit Client Script performs a final client-side check before save. An onCellEdit Client Script handles a different interaction path, namely list editing.

This approach avoids unnecessary complexity. Since the required behavior is local to the Incident user interface, introducing an external application or separate service would increase the project size without adding value to the stated requirements.

4.2 Proposed Solution

When Impact is High, Assignment group is mandatory.

When Impact is High, Urgency is read-only.

When Impact changes to High, Urgency is automatically set to High.

A High Impact Incident cannot be saved when Assigned To is empty.

State cannot be changed using Incident list editing.

State can still be changed through the Incident form.

The proposed solution is designed to be easy to demonstrate. First, an Incident is opened. Next, Impact is changed to High. The user can immediately observe the mandatory Assignment group and read-only Urgency behavior. The Urgency value is also automatically updated.

The second part of the demonstration is validation. If Assigned To is empty and the user attempts to save, the onSubmit Client Script prevents the save and displays an error. If Assigned To is filled, the same save operation can proceed.

The final part demonstrates list editing. The State field is selected from an Incident list and the edit is rejected. When the same State is changed through the Incident form, the update is allowed. This proves that the restriction is specifically applied to list editing.

4.3 Solution Architecture

User Interaction → ServiceNow Incident Form/List → UI Policies + Client Scripts → Field Control / Validation → Incident Record

The architecture is lightweight and is implemented within the ServiceNow platform without a separate external backend or database.

The architecture contains the user interaction layer, the ServiceNow interface, the client-side rule layer, and the Incident record. The user interacts with the form or list. ServiceNow then executes the appropriate configuration or script according to the event.

The architecture is event-driven. The onChange script reacts to a field change, the onSubmit script reacts to a save attempt, and the onCellEdit script reacts to a list edit. The UI Policy is condition-driven and changes field behavior when its condition is satisfied.

Because the rules are separated by purpose, maintenance is straightforward. A change to the mandatory behavior can be handled in the UI Policy Action, while a change to automatic Urgency assignment can be handled in the onChange Client Script.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Planned story points: Sprint 1 = 10, Sprint 2 = 11, Total = 21. Proposed velocity based on the two planned sprints = 10.5 story points/sprint.

Actual team members, sprint dates, and completed story points were not specified in the supplied project material.

Sprint	Functional Requirement / Task	Story Points	Priority
Sprint 1	Create High Impact Control UI Policy	3	High
Sprint 1	Make Assigned To mandatory for High Impact	2	High
Sprint 1	Make Urgency read-only for High Impact	2	High
Sprint 1	Auto-set Urgency using onChange Client Script	3	High
Sprint 2	Prevent save when Assigned To is missing	3	High
Sprint 2	Prevent State changes through list editing	3	High
Sprint 2	Test validation and field behavior	5	High

Sprint 1 concentrates on the main High Impact form behavior. The first sprint establishes the UI Policy, field actions, and automatic Urgency update. These functions form the basic user experience of the project.

Sprint 2 concentrates on validation, list behavior, and testing. The save validation makes the data-entry process stricter, while the onCellEdit rule ensures that the intended State control is maintained when users work from a list.

The story-point distribution reflects the relative work required for the planned activities. Testing receives a larger allocation because every rule must be checked under both valid and invalid conditions.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The project uses simple client-side operations, and the expected response is immediate during normal Incident form and list interactions.

The available testing is primarily functional because the project requirements concern field behavior and user interaction. The expected response is immediate from the user's perspective: fields change state, values are set, validation messages appear, and list edits are accepted or rejected.

Performance should be understood here as normal client-side responsiveness rather than as a benchmark. The project does not introduce a separate service or heavy processing step. The configured rules use simple field conditions and client-side operations.

For a formal production deployment, additional performance testing could be performed in the target ServiceNow instance with representative data and realistic user activity. Such measurements are outside the supplied project scope, so they are not invented in this report.

Test ID	Scenario	Expected Result	Result
TC-001	Impact = High	Assignment group becomes mandatory	Pass
TC-002	Impact = High	Urgency becomes read-only	Pass
TC-003	Change Impact to High	Urgency automatically becomes High	Pass
TC-004	High Impact + Assigned To empty + Save	Save is blocked with error	Pass
TC-005	High Impact + Assigned To filled + Save	Record saves successfully	Pass
TC-006	High → Medium	Restrictions are reversed; Urgency	Pass

		editable	
TC-007	Edit State from list	Alert appears; edit rejected	Pass
TC-008	Edit State from form	State update allowed	Pass

The test cases cover both expected restrictions and expected permissions. This is important because a rule is not considered fully correct if it only blocks an invalid action; it must also permit the valid action.

TC-001 and TC-002 verify the UI Policy. TC-003 verifies the automatic Client Script behavior. TC-004 and TC-005 verify the save validation in both negative and positive cases. TC-006 verifies that the High Impact restrictions are conditional. TC-007 and TC-008 verify the difference between list editing and form editing.

7. RESULTS

7.1 Output Screenshots

Screenshots of the completed ServiceNow instance can be placed here to show the expected outputs.

Screenshot 1 – High Impact UI Policy / Assignment group mandatory

The first expected output shows an Incident after Impact is set to High. The Assignment group field should visually indicate that it is mandatory. This demonstrates that the UI Policy condition has been evaluated correctly.

Screenshot 2 – Urgency read-only for High Impact

The second expected output shows the Urgency field in a non-editable state while the Incident remains High Impact. This confirms the read-only UI Policy Action.

Screenshot 3 – Automatic Urgency update

The third expected output demonstrates the onChange Client Script. When Impact changes to High, the Urgency field is automatically populated with High without requiring the user to type the value manually.

Screenshot 4 – Save blocked when Assigned To is missing

The fourth expected output shows the validation message generated when a High Impact Incident is saved without Assigned To. The important result is that the record is not submitted until the required information is provided.

Screenshot 5 – State list edit blocked

The fifth expected output shows the message or rejection behavior when a user attempts to edit State directly from the Incident list. The form-based State change remains available.

The overall result is a more controlled Incident-entry experience. The user receives immediate feedback, important values are protected, and invalid actions are stopped before they create an unwanted record state.

Component	**Script / Configuration**
UI Policy	High Impact Control

UI Policy Action	Assignment group – mandatory
UI Policy Action	Urgency – read-only
onChange Client Script	Auto set urgency for high impact
onSubmit Client Script	Prevent save if Assigned To missing
onCellEdit Client Script	Prevent state change via list edit

8. ADVANTAGES & DISADVANTAGES

Advantages

Improves Incident data consistency.

Provides dynamic field control.

Reduces invalid submissions.

Automates Urgency assignment for High Impact Incidents.

Prevents inappropriate list-based State changes.

Provides immediate feedback during Incident entry.

Uses native ServiceNow configuration and client-side scripting.

Separates different behaviors into understandable rules.

The main advantage is consistency. When the same condition is encountered repeatedly, the system applies the same behavior instead of depending entirely on individual user memory.

Another advantage is usability. Mandatory and read-only field behavior gives users a visual indication of what is expected. Automatic Urgency assignment also reduces repetitive manual entry.

The solution is also maintainable because the requirements are divided into UI Policies and Client Scripts. Each rule has a clear purpose and can be tested separately.

Disadvantages

Rules are specific to the configured Incident fields and conditions.

Client-side behavior depends on the ServiceNow form/list interaction.

The supplied project scope does not include server-side validation or performance benchmarking.

Changes to field names, values, or business rules may require corresponding configuration updates.

Users working through different interfaces may require additional validation depending on the deployment design.

The main limitation is that the project focuses on client-side behavior. For a larger production solution, important business rules may also need server-side enforcement so that they remain effective across all relevant data-entry paths.

Another limitation is that the project is intentionally narrow. It handles the specified High Impact and State-editing scenarios rather than attempting to solve every possible Incident-data quality problem.

9. CONCLUSION

The project demonstrates how ServiceNow UI Policies and Client Scripts can be combined to enforce dynamic field behavior, automate values, validate Incident submissions, and control list editing. The implemented rules support cleaner and more consistent Incident records.

The project begins with a simple problem: users may create or modify Incident records without following all required data rules. The solution places controls directly into the Incident user experience. When Impact becomes High, the form changes immediately. Assignment group becomes mandatory, Urgency becomes read-only, and the Urgency value is automatically set to High.

The onSubmit Client Script adds another layer of protection by checking Assigned To before a High Impact Incident is saved. This prevents a record from being submitted when an important field is missing. The onCellEdit Client Script handles the list interaction separately and prevents State changes through direct list editing.

The test cases show that the solution is not only restrictive. It also supports valid actions. A High Impact Incident with Assigned To filled can be saved, the special restrictions can be reversed when Impact changes, and State can still be changed through the Incident form.

Overall, the project is a practical example of simple ServiceNow customization. It does not require a complex architecture, but it demonstrates important concepts such as conditional field control, event-based scripting, client-side validation, and user-interface consistency.

10. FUTURE SCOPE

Add additional validation rules for other Incident fields.

Extend automation to more Impact/Urgency combinations.

Add role-based controls where required.

Introduce broader automated test coverage.

Add reporting or dashboards to monitor validation outcomes.

Future scope can extend the current project without changing its basic approach. Additional Incident fields can be controlled using the same combination of UI Policies and Client Scripts where appropriate.

The automation can also be expanded beyond the single High Impact condition. Different combinations of Impact, Urgency, Priority, Category, or Assignment information could be supported if required by the business process.

Role-based controls could provide different behavior for different categories of users. For example, selected users may be allowed to perform actions that are restricted for normal users, depending on the organization's ServiceNow configuration.

Testing can also be expanded with more positive and negative scenarios. Automated testing would help ensure that future changes do not accidentally break existing behavior.

Finally, reporting or dashboards could be added to help administrators understand how often validation rules are triggered. This could turn the project from a simple data-entry control into a broader data-quality monitoring solution.

11. APPENDIX

Source Code / Scripts

Project Demo Link: Not specified in the supplied ServiceNow project source.

GitHub Link: Not specified.

Dataset Link: Not applicable / not specified.

Document prepared: 30 September 2026

Configuration Type	Name
UI Policy	High Impact Control
UI Policy Action	Assignment group mandatory
UI Policy Action	Urgency read-only
Client Script – onChange	Auto set urgency for high impact
Client Script – onSubmit	Prevent save if Assigned To missing
Client Script – onCellEdit	Prevent state change via list edit